

CLASSIFICAÇÃO DE CARROCERIAS DE AUTOMÓVEIS

META I – INTELIGÊNCIA COMPUTACIONAL



Pedro Saraiva – 2023146226
Daniel Soares – 2023144661

OBJETIVOS E POTENCIAIS APLICAÇÕES REAIS

-
1. **Gestão de estacionamento** → otimização das vagas e verificação automática se o tipo de veículo é permitido em determinado espaço
 2. **Seguros automóveis** → identificação automática do tipo de carro em fotos enviadas por clientes
 3. **Portagens inteligentes** → deteção da categoria do veículo para cobrança automática do valor correto
 4. **Controlo de acessos no campus do ISEC** → identificação automática do tipo de carro, para limitar o número de carros de cada categoria, garantindo uma distribuição equilibrada no estacionamento

CLASSES



CONVERTIBLE



COUPE



HATCHBACK



SUV



SEDAN

DATASETS

- **Nome:** “Car Body Types Images Dataset” e “Stanford Car Body Type Data”
- **Fonte:** <https://www.kaggle.com/datasets/ademboukhris/cars-body-type-cropped/data> e <https://www.kaggle.com/datasets/mayurmahurkar/stanford-car-body-type-data>

Conjunto	Nº total de imagens	Distribuição por classe
Treino	8.000	Convertible: 1.759; Coupé: 1.594; Hatchback: 1.130; Sedan: 1.758; SUV: 1.759
Validação	1.000	200 por classe
Teste	1.000	200 por classe

CONFIGURAÇÃO DO AMBIENTE E BIBLIOTECAS UTILIZADAS

O projeto foi desenvolvido em **Python 3.10.18**, utilizando as seguintes bibliotecas principais:

- **TensorFlow com API Keras** – para criação, treino e avaliação dos modelos de rede neuronal (CNN e MLP).
- **Pickle** – para guardar o histórico de treino (history.pkl). **NumPy** – para manipulação matrizes e processamento de previsões.
- **Matplotlib** – para visualização de resultados e geração da matriz de confusão de vetores,.
- **scikit-learn** – para cálculo de métricas estatísticas: *accuracy*, *precision*, *recall* (*sensibilidade*), *f1-score*, *AUC* e *especificidade*.

A execução foi feita em ambiente local com GPU (quando possível), garantindo tempos de treino mais rápidos.

TREINO

PRÉ-PROCESSAMENTO E NORMALIZAÇÃO DAS IMAGENS

```
# pre-processamento e normalizacao das imagens
def ds(path, training):
    d = keras.preprocessing.image_dataset_from_directory(
        path, image_size=(128, 128), batch_size=32,
        label_mode="categorical", shuffle=training
    )
    return d.map(lambda x,y: (tf.cast(x, tf.float32)/255.0, y)).prefetch(tf.data.AUTOTUNE)

# carregar dataset
train_ds = ds("../dataset/train", True)
val_ds = ds("../dataset/validation", False)
```

```
✓ DATASET
  ✓ test
    > Convertible
    > Coupe
    > Hatchback
    > Sedan
    > SUV
    > train
    > validation
```

MODELO BASE – ARQUITETURA CNN

```
# arquitetura do modelo CNN
model = keras.Sequential([
    layers.Input((128,128,3)),

    layers.Conv2D(32,(3,3),activation="relu"), layers.MaxPooling2D(4,4),
    layers.Conv2D(64,(3,3),activation="relu"), layers.MaxPooling2D(4,4),
    layers.Conv2D(128,(3,3),activation="relu"),

    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.5),
    layers.Dense(5, activation="softmax")
])
```

MODELO ALTERNATIVO – ARQUITETURA MLP

```
# arquitetura do modelo MLP
model = keras.Sequential([
    layers.Input((128, 128, 3)),
    layers.Flatten(),

    layers.Dense(32, activation="relu"), layers.BatchNormalization(),
    layers.Dense(64, activation="relu"), layers.BatchNormalization(),
    layers.Dense(128, activation="relu"), layers.BatchNormalization(),

    layers.Dropout(0.5),
    layers.Dense(5, activation="softmax")
])
```

OTIMIZADOR E PARÂMETROS DE TREINO DOS MODELOS CNN E MLP

```
# earlyStopping
earlyStopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)

# parametros de treino
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"])

# treino
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=100,
    callbacks=[earlyStopping]
)
```

No final do treino são salvos dois ficheiros: um com o melhor modelo do treino; e outro com o histórico do treino (métricas de loss e accuracy do treino e validação).

RAZÃO PARA ESCOLHA DO KERAS

- Em primeiro lugar, o Keras permite que o modelo seja treinado utilizando a GPU, o que acelera significativamente o processo de treino.
- Principalmente no modelo MLP:
 - O Keras oferece um maior controlo e flexibilidade na definição da rede neuronal, permitindo configurar manualmente todas as camadas e ajustar a sua estrutura de forma a aproximar-se ao máximo do modelo CNN desenvolvido. Desta forma, torna-se possível realizar uma comparação direta entre ambos os modelos, algo que não seria possível utilizando a biblioteca *scikit-learn* (abordagem adotada nas aulas práticas).

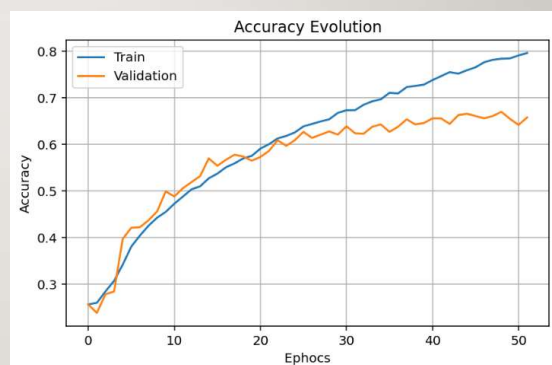
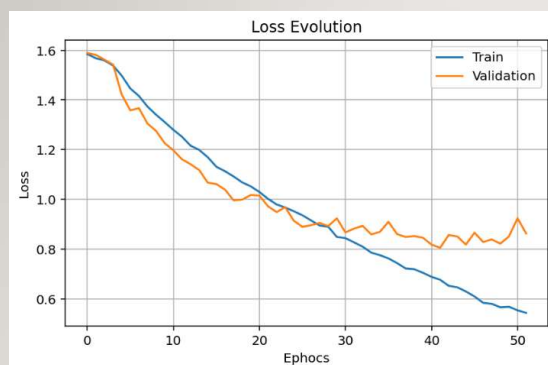
TESTE

No teste foram feitos os seguintes procedimentos:

- Pré-processamento e normalização das imagens (tal como no treino).
- Previsões a partir do melhor modelo salvo no treino nas imagens para teste.
- Apresentado a Matriz de confusão.
- Apresentado o resultado das seguintes métricas:
 - Accuracy;
 - Precisão;
 - Recall (sensibilidade);
 - F1-score (f-measure);
 - Especificidade por classe;
 - AUC;

ANÁLISE DE RESULTADOS

ANÁLISE DE RESULTADOS – CONJUNTO DE TREINO (CNN)



ANÁLISE DE RESULTADOS – CONJUNTO DE TESTE (CNN)

Confusion matrix

	Convertible	Coupe	Hatchback	SUV	Sedan
Actual					
Convertible	167	15	6	5	7
Coupe	34	98	15	5	48
Hatchback	12	14	117	18	39
SUV	6	1	18	165	10
Sedan	9	40	21	15	115
	Convertible	Coupe	Hatchback	SUV	Sedan
	Predicted				

Found 1000 files belonging to 5 classes.

Accuracy: 0.662

Relatório (precision, recall(sensibilidade), F1):

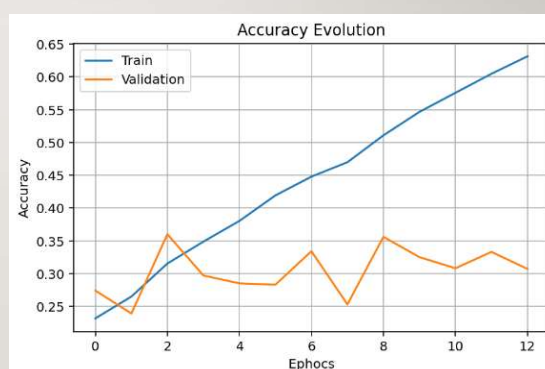
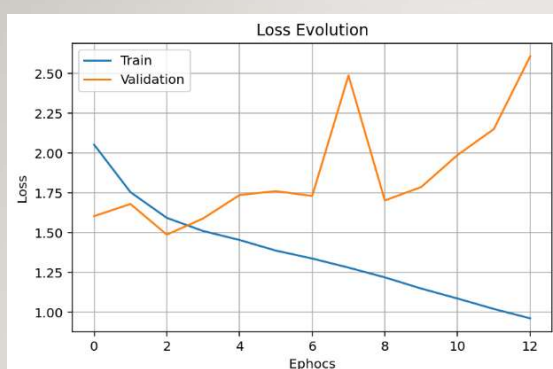
	precision	recall	f1-score	support
Convertible	0.7325	0.8350	0.7804	200
Coupe	0.5833	0.4900	0.5326	200
Hatchback	0.6610	0.5850	0.6207	200
SUV	0.7933	0.8250	0.8088	200
Sedan	0.5251	0.5750	0.5489	200
accuracy			0.6620	1000
macro avg	0.6590	0.6620	0.6583	1000
weighted avg	0.6590	0.6620	0.6583	1000

Especificidade por classe:

Convertible: 0.9237
 Coupe: 0.9125
 Hatchback: 0.9250
 SUV: 0.9463
 Sedan: 0.8700

Macro AUC (OvR): 0.8967

ANÁLISE DE RESULTADOS – CONJUNTO DE TREINO (MLP)



ANÁLISE DE RESULTADOS – CONJUNTO DE TESTE (MLP)

Confusion matrix

Actual	Convertible	91	26	18	23	42
	Coupe	53	28	33	36	50
	Hatchback	39	25	54	30	52
	SUV	23	14	18	95	50
	Sedan	52	18	22	37	71
		Convertible	Coupe	Hatchback	SUV	Sedan
		Predicted				

Found 1000 files belonging to 5 classes.

Accuracy: 0.339

Relatório (precision, recall(sensibilidade), F1):

	precision	recall	f1-score	support
Convertible	0.3527	0.4550	0.3974	200
Coupe	0.2523	0.1400	0.1801	200
Hatchback	0.3724	0.2700	0.3130	200
SUV	0.4299	0.4750	0.4513	200
Sedan	0.2679	0.3550	0.3054	200
accuracy			0.3390	1000
macro avg	0.3350	0.3390	0.3294	1000
weighted avg	0.3350	0.3390	0.3294	1000

Especificidade por classe:

Convertible: 0.7913
 Coupe: 0.8962
 Hatchback: 0.8862
 SUV: 0.8425
 Sedan: 0.7575

Macro AUC (OvR): 0.6631

COMPARAÇÃO ENTRE CNN E MLP

- Os resultados práticos são claros: a CNN atingiu 66,2 % de accuracy e um AUC de 0,8967, enquanto a MLP obteve apenas 33,9 % de accuracy e AUC de 0,6631.
- A rede neuronal MLP começa por achatar a imagem ($128 \times 128 \times 3 \rightarrow 49\ 152$ valores) e, ao fazê-lo, perde a informação espacial, isto é, a relação entre píxeis vizinhos. Na prática, trata cada píxel como uma unidade independente.
- A CNN, por outro lado, opera com filtros bidimensionais (2D) que exploram vizinhanças locais (receptive fields), permitindo captar arestas, curvas e padrões de forma consistente no espaço. Esta capacidade de preservar a estrutura espacial é fundamental na classificação de imagens de carros, onde a forma e a disposição dos elementos visuais são determinantes.

CONCLUSÕES

- Inicialmente, a rede apresentava uma elevada accuracy no treino, mas um desempenho significativamente inferior na validação, o que indicava uma falta de generalização. A principal dificuldade encontrada foi o controlo do overfitting nas primeiras versões do modelo. A introdução de técnicas como Dropout, EarlyStopping e GlobalAveragePooling permitiu mitigar esse problema, estabilizando o treino e garantindo resultados mais consistentes e equilibrados entre os diferentes conjuntos de dados.
- Na meta final, o modelo CNN demonstrou uma boa estabilidade e generalização, com curvas de treino e validação consistentes. A análise das métricas confirmou a eficácia da arquitetura implementada, validando a escolha da CNN como modelo base para continuação do trabalho nas próximas metas.